

Dokumentasi Project

# Backend Wisata Puncak Mas

*Fitur Auth — NestJS*



Dibuat Oleh:

**Muhammad Tammam**

NPM: 23312202

Kelas: IF 23 E

Universitas Teknokrat Indonesia

## 14. Gambaran Umum Fitur Auth

Fitur Auth (Authentication) adalah sistem yang bertanggung jawab untuk mengelola identitas admin aplikasi. Sistem ini memungkinkan pendaftaran akun admin baru dan proses masuk (login) yang aman menggunakan kombinasi username dan password. Setelah login berhasil, server akan menerbitkan sebuah token JWT (JSON Web Token) yang menjadi bukti bahwa admin tersebut telah terautentikasi dan berhak melakukan operasi-operasi yang dilindungi.

### Teknologi Tambahan yang Digunakan

| Teknologi        | Kegunaan                                                                                                                                                              |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| @nestjs/jwt      | Library resmi NestJS untuk membuat dan memverifikasi JWT. Digunakan untuk menerbitkan access_token setelah login berhasil.                                            |
| @nestjs/passport | Integrasi Passport.js ke dalam NestJS. Mendaftarkan strategi autentikasi default (JWT) yang akan digunakan di seluruh aplikasi.                                       |
| bcrypt           | Library untuk melakukan hashing password. Password tidak pernah disimpan dalam bentuk teks biasa di database — selalu dalam bentuk hash yang tidak bisa dikembalikan. |

## 15. Struktur File Fitur Auth (folder src/auth/)

Seluruh kode fitur autentikasi diorganisir di dalam folder **src/auth/** dengan pembagian tanggung jawab yang jelas pada setiap file:

| File / Folder                     | Fungsi Singkat                                                                                                                                    |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| src/auth/auth.module.ts           | Pusat pendaftaran fitur auth. Mendaftarkan repository Admin, mengonfigurasi JWT (secret key dan masa berlaku), serta mendaftarkan PassportModule. |
| src/auth/auth.controller.ts       | Mengatur alamat (endpoint) API untuk fitur auth, yaitu POST /auth/register dan POST /auth/login.                                                  |
| src/auth/auth.service.ts          | Tempat seluruh logika bisnis autentikasi dijalankan: validasi username, hashing password, pembuatan JWT, dan verifikasi login.                    |
| src/auth/entities/admin.entity.ts | Gambaran struktur tabel 'admins' di database. Mendefinisikan kolom id, username (unik), dan password.                                             |
| src/auth/dto/login.dto.ts         | Aturan validasi data yang dikirim saat register maupun login. Digunakan bersama oleh kedua endpoint karena formatnya sama.                        |

## 16. Struktur Data di Database (admin.entity.ts)

File entity mendefinisikan tabel **admins** di database. TypeORM akan membuat tabel ini secara otomatis berdasarkan dekorator yang ada di file tersebut. Nama tabel di database adalah **admins** (jamak, huruf kecil), sesuai dekorator **@Entity('admins')**.

| Kolom di Database | Nama di Kode | Tipe Data | Keterangan                                                                                                                       |
|-------------------|--------------|-----------|----------------------------------------------------------------------------------------------------------------------------------|
| id                | id           | INT       | Nomor unik yang diisi otomatis oleh database (auto increment). Berfungsi sebagai identitas setiap baris data.                    |
| username          | username     | VARCHAR   | Nama pengguna admin. Diberi constraint UNIQUE sehingga tidak boleh ada dua admin dengan username yang sama.                      |
| password          | password     | VARCHAR   | Password admin yang disimpan dalam bentuk hash bcrypt, bukan teks biasa. Panjang kolom disesuaikan otomatis dengan panjang hash. |

**Catatan Keamanan Penting:** Kolom **password** tidak pernah menyimpan teks asli yang diketik oleh admin. Sebelum disimpan, password selalu diproses terlebih dahulu oleh bcrypt menjadi sebuah hash acak. Proses ini bersifat satu arah — hash tidak bisa diubah kembali menjadi password asli, sehingga data admin tetap aman meskipun database bocor.

## 17. Aturan Input Data Auth (LoginDto)

Fitur auth menggunakan satu DTO (Data Transfer Object) yang sama untuk endpoint register maupun login, yaitu **LoginDto**. Hal ini dimungkinkan karena kedua endpoint menerima data dengan format yang identik: username dan password. Jika salah satu field tidak dikirim atau bukan bertipe string, server akan langsung menolak request dengan pesan error validasi.

| Field    | Tipe Data | Aturan Validasi                 | Keterangan                                                                           |
|----------|-----------|---------------------------------|--------------------------------------------------------------------------------------|
| username | string    | Wajib diisi. Harus berupa teks. | Nama pengguna admin. Tidak boleh kosong.                                             |
| password | string    | Wajib diisi. Harus berupa teks. | Password admin. Tidak boleh kosong. Akan di-hash sebelum disimpan (khusus register). |

## 18. Konfigurasi Modul Auth (auth.module.ts)

File `auth.module.ts` adalah pusat konfigurasi untuk seluruh fitur autentikasi. Di sinilah tiga hal penting dikonfigurasi agar fitur auth bisa berjalan dengan benar:

| Konfigurasi                           | Nilai                                                 | Keterangan                                                                                                                                              |
|---------------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| TypeOrmModule<br>.forFeature([Admin]) | Entity Admin                                          | Mendaftarkan repository Admin ke dalam module ini sehingga AuthService bisa menggunakan adminRepository untuk membaca dan menulis data ke tabel admins. |
| PassportModule<br>.register(...)      | defaultStrategy: 'jwt'                                | Mendaftarkan Passport.js dengan strategi default JWT. Ini mempersiapkan infrastruktur agar guard berbasis JWT bisa digunakan di modul lain nantinya.    |
| JwtModule .register(...)              | secret: 'KUNCI_<br>RAHASIA_ADMIN',<br>expiresIn: '1d' | Mengonfigurasi pembuatan JWT. Secret key digunakan untuk menandatangani token, sedangkan expiresIn '1d' berarti token akan kedaluwarsa dalam 1 hari.    |

**Catatan Keamanan:** Nilai **secret** pada JwtModule ('KUNCI\_RAHASIA\_ADMIN') sebaiknya tidak ditulis langsung di dalam kode (hardcoded). Pada environment production, nilai ini harus dipindahkan ke file **.env** dan diakses melalui ConfigModule agar tidak terekspos jika kode sumber diunggah ke repositori publik.

## 19. Daftar Endpoint API Auth (auth.controller.ts)

Controller auth mengatur dua endpoint yang semuanya bisa diakses tanpa kunci akses (bersifat publik). Semua endpoint auth dimulai dari alamat dasar **/auth**.

| Method | Alamat         | Kunci Akses? | Hasil jika Berhasil                     | Fungsi                                                                           |
|--------|----------------|--------------|-----------------------------------------|----------------------------------------------------------------------------------|
| POST   | /auth/register | Tidak        | 201 + pesan 'Registrasi Admin berhasil' | Membuat akun admin baru. Password di-hash sebelum disimpan ke database.          |
| POST   | /auth/login    | Tidak        | 200 + { access_token: '...' }           | Memverifikasi username dan password. Jika valid, mengembalikan JWT access_token. |

## 20. Logika Utama Fitur Auth (auth.service.ts)

AuthService adalah inti dari fitur autentikasi. Di sinilah semua proses kritis dijalankan, mulai dari validasi keunikan username, hashing password, hingga penerbitan token JWT.

### 20.1 Registrasi Admin Baru (register)

Proses registrasi berjalan melalui empat langkah berurutan:

| Langkah                        | Penjelasan                                                                                                                                                                                                                                                     |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Cek keunikan username       | Service mencari data di tabel admins menggunakan username yang dikirim. Jika sudah ada, proses langsung dihentikan dan server mengirimkan error 409 Conflict dengan pesan 'Username admin sudah digunakan'.                                                    |
| 2. Buat salt dan hash password | bcrypt.genSalt() menghasilkan nilai acak (salt) yang unik. Salt ini kemudian digabungkan dengan password asli oleh bcrypt.hash() untuk menghasilkan hash. Dua admin dengan password yang sama pun akan menghasilkan hash yang berbeda karena salt-nya berbeda. |
| 3. Simpan data admin baru      | Objek admin baru dibuat menggunakan adminRepository.create() lalu disimpan ke database dengan adminRepository.save(). Password yang tersimpan adalah hash, bukan teks asli.                                                                                    |
| 4. Kembalikan respons sukses   | Server mengembalikan objek JSON { message: 'Registrasi Admin berhasil' } dengan status HTTP 201 Created.                                                                                                                                                       |

#### Contoh kode kunci pada proses register:

```
const salt = await bcrypt.genSalt();
const hashedPassword = await bcrypt.hash(password, salt);
// Password yang tersimpan ke DB adalah hashedPassword, bukan password asli
```

## 20.2 Login Admin (login)

Proses login memverifikasi identitas admin dan menerbitkan token akses:

| Langkah                            | Penjelasan                                                                                                                                                                                                                                                                        |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Cari admin berdasarkan username | Service mencari data admin di database menggunakan username yang dikirim. Jika tidak ditemukan, proses langsung dihentikan (lihat langkah 3).                                                                                                                                     |
| 2. Verifikasi password             | bcrypt.compare() membandingkan password teks biasa yang dikirim dengan hash yang tersimpan di database. Fungsi ini secara internal melakukan hashing ulang dan membandingkan hasilnya — password asli tidak pernah disimpan atau dikembalikan.                                    |
| 3. Tangani kegagalan               | Jika username tidak ditemukan ATAU password salah, server mengembalikan error 401 Unauthorized dengan pesan 'Username atau password Admin salah'. Kedua kondisi sengaja digabung dalam satu pesan untuk mencegah penyerang menebak apakah username tersebut terdaftar atau tidak. |
| 4. Buat dan kembalikan JWT         | Jika username dan password cocok, service membuat payload JWT berisi { id, username, role: 'admin' } lalu menandatangani menggunakan jwtService.sign(). Token ini dikembalikan ke frontend sebagai { access_token: '...' }.                                                       |

#### Contoh kode kunci pada proses login:

```
const payload = { id: admin.id, username: admin.username, role: 'admin' };
const accessToken = this.jwtService.sign(payload);
```

```
return { access_token: accessToken };
```

## 21. Contoh Request dan Response

### 21.1 Register Admin Baru

#### Request:

```
POST /auth/register
Content-Type: application/json

{
  "username": "admin_puncakmas",
  "password": "password123"
}
```

| Kondisi             | HTTP Status     | Response Body                                        |
|---------------------|-----------------|------------------------------------------------------|
| Berhasil            | 201 Created     | { "message": "Registrasi Admin berhasil" }           |
| Username sudah ada  | 409 Conflict    | { "message": "Username admin sudah digunakan" }      |
| Field tidak lengkap | 400 Bad Request | { "message": ["username should not be empty", ...] } |

### 21.2 Login Admin

#### Request:

```
POST /auth/login
Content-Type: application/json

{
  "username": "admin_puncakmas",
  "password": "password123"
}
```

| Kondisi                 | HTTP Status      | Response Body                                        |
|-------------------------|------------------|------------------------------------------------------|
| Berhasil                | 200 OK           | { "access_token": "eyJhbGciOi..." }                  |
| Username/password salah | 401 Unauthorized | { "message": "Username atau password Admin salah" }  |
| Field tidak lengkap     | 400 Bad Request  | { "message": ["username should not be empty", ...] } |

## 22. Alur Kerja Sistem Auth (Request Flow)

### 22.1 Alur Registrasi Admin Baru

| Langkah | Siapa yang Bertugas  | Apa yang Terjadi                                                                                                          |
|---------|----------------------|---------------------------------------------------------------------------------------------------------------------------|
| 1       | Frontend             | Mengirim POST /auth/register dengan body { username, password }.                                                          |
| 2       | ValidationPipe       | Server memeriksa format data. Jika username atau password kosong/bukan string, request langsung ditolak dengan error 400. |
| 3       | AuthController       | Request diterima dan diteruskan ke method register() di AuthService.                                                      |
| 4       | AuthService          | Cek apakah username sudah ada di tabel admins. Jika ada, lempar error 409 Conflict.                                       |
| 5       | AuthService + bcrypt | Jika username belum ada: buat salt, hash password menggunakan bcrypt.                                                     |
| 6       | TypeORM Repository   | Simpan objek admin baru (dengan hashed password) ke tabel admins di database.                                             |
| 7       | Response ke Frontend | Kembalikan { message: 'Registrasi Admin berhasil' } dengan status 201.                                                    |

### 22.2 Alur Login Admin

| Langkah | Siapa yang Bertugas  | Apa yang Terjadi                                                                                                                       |
|---------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 1       | Frontend             | Mengirim POST /auth/login dengan body { username, password }.                                                                          |
| 2       | ValidationPipe       | Server memeriksa format data. Jika tidak valid, request ditolak dengan error 400.                                                      |
| 3       | AuthController       | Request diterima dan diteruskan ke method login() di AuthService.                                                                      |
| 4       | AuthService          | Cari admin berdasarkan username di database.                                                                                           |
| 5       | AuthService + bcrypt | Jika admin ditemukan: bandingkan password yang dikirim dengan hash di database menggunakan bcrypt.compare().                           |
| 6       | AuthService + JWT    | Jika password cocok: buat payload { id, username, role } dan tandatangi menggunakan jwtService.sign() untuk menghasilkan access_token. |
| 7       | Response ke Frontend | Kembalikan { access_token: '...' } dengan status 200. Jika username atau password salah, kembalikan error 401.                         |

## 23. Pengujian Otomatis (Unit Test)

Sebagaimana modul wisata, modul auth juga sebaiknya dilengkapi dengan file-file pengujian otomatis menggunakan framework Jest. Berikut adalah pengujian yang perlu ditambahkan:

| File Pengujian          | Komponen yang Diuji | Yang Diperiksa                                                                                                            |
|-------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------|
| auth.controller.spec.ts | AuthController      | Memastikan controller berhasil dibuat dan endpoint register serta login terdaftar dengan benar.                           |
| auth.service.spec.ts    | AuthService         | Memastikan service berhasil dibuat dan tidak error saat diinisialisasi bersama dependency-nya (repository & JWT service). |

Untuk menjalankan semua pengujian termasuk modul auth, gunakan perintah yang sama:

```
npm run test
```

## 24. Cara Menggunakan Token JWT di Frontend

Setelah berhasil login, frontend akan menerima **access\_token**. Token ini harus disertakan pada setiap request yang membutuhkan autentikasi (seperti POST, PATCH, DELETE pada endpoint /wisata yang dilindungi WisataGuard).

**Cara menyertakan token pada header request:**

```
// Contoh penggunaan di frontend (fetch API)
fetch('http://localhost:3000/wisata', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer eyJhbGci...' // Token dari response login
  },
  body: JSON.stringify({ ... })
});
```

**Catatan:** Token JWT memiliki masa berlaku **1 hari (1d)** sesuai konfigurasi di auth.module.ts. Setelah kedaluwarsa, frontend harus meminta token baru dengan melakukan login ulang.